Penguin Regression Walkthrough

This folder contains an example of two common prediction models in R using the `palmerpenguins` dataset:

1. A **linear regression** model to predict a penguin's `body_mass_g`
2. A **logistic regression** model to predict whether a penguin is `female` or `male`

The R script for this example is located here:

- Regressions/penguin_regressions.R

What dataset are we using?

We are using the `penguins` dataset from the `palmerpenguins` package. This dataset includes measurements for penguins such as:

- `body_mass_g`: the penguin's body mass in grams
- `bill_length_mm`: bill length in millimeters
- `bill_depth_mm`: bill depth in millimeters
- `flipper_length_mm`: flipper length in millimeters
- `sex`: whether the penguin is male or female

This is a great practice dataset because it has both:

- **numeric variables** we can predict with linear regression
- a **category variable** we can predict with logistic regression

Important note about the variable names

The original request used the names `boddy_mass_g` and `bill_length_m`, but in the actual dataset the correct column names are:

- `body_mass_g`
- `bill_length_mm`

This is very common in real data work. Before building a model, always check the actual variable names in the dataset.

Step 1: Load the packages

The script starts with:

```
library(tidyverse)
library(palmerpenguins)
```

Why do we need these?

- `tidyverse` gives us tools like `select()`, `drop_na()`, and `mutate()`
- `palmerpenguins` gives us access to the `penguins` dataset

Step 2: Clean the data

The script creates a cleaned dataset called `penguins_clean`:

```
penguins_clean <- penguins |>
  select(body_mass_g, bill_length_mm, bill_depth_mm, flipper_length_mm,
sex) |>
  drop_na() |>
  mutate(
    sex = str_to_lower(sex),
    sex = factor(sex, levels = c("female", "male"))
  )
```

Let's break this down.

`select(...)`

This keeps only the variables we want to use in the models. That helps us focus only on the columns needed for prediction.

`drop_na()`

Some rows in the dataset have missing values. Regression models usually cannot use rows with missing values in the predictor or outcome columns. `drop_na()` removes those incomplete rows.

`mutate(...)`

This makes sure the `sex` variable is in a clean format:

- `str_to_lower(sex)` changes values to lowercase
- `factor(sex, levels = c("female", "male"))` turns `sex` into a categorical variable with an explicit order

That last step matters because logistic regression needs to know the categories it is modeling.

Step 3: Build the linear regression model

The script fits this model:

```
body_mass_model <- lm(
  body_mass_g ~ bill_length_mm + bill_depth_mm + flipper_length_mm,
  data = penguins_clean
)
```

What is linear regression?

Linear regression is used when the outcome we want to predict is **numeric**.

Here, the outcome is:

- `body_mass_g`

The predictors are:

- `bill_length_mm`
- `bill_depth_mm`
- `flipper_length_mm`

In simple terms, linear regression tries to find the best equation for predicting body mass from these body measurements.

The model is trying to learn an equation like this:

```
predicted body mass =
intercept +
(bill length coefficient × bill length) +
(bill depth coefficient × bill depth) +
(flipper length coefficient × flipper length)
```

Results of the linear regression

When the script was run, the model produced these key results:

- Intercept: about `-6445.48`
- `bill_length_mm` coefficient: about `3.29`
- `bill_depth_mm` coefficient: about `17.84`
- `flipper_length_mm` coefficient: about `50.76`
- Multiple R-squared: about `0.7639`

How do we interpret those numbers?

Intercept

The intercept is the predicted value when all predictor variables are zero. In this case, that is not a realistic penguin measurement, so the intercept is mainly a mathematical part of the model rather than something biologically meaningful.

Coefficients

Each coefficient tells us how the predicted body mass changes when one predictor increases by 1 unit, while the other predictors stay the same.

- A 1 mm increase in `bill_length_mm` is associated with about a **3.29 gram increase** in predicted body mass
- A 1 mm increase in `bill_depth_mm` is associated with about a **17.84 gram increase** in predicted body mass

- A 1 mm increase in `flipper_length_mm` is associated with about a **50.76 gram increase** in predicted body mass

Among these predictors, `flipper_length_mm` has the strongest relationship with body mass in this model.

P-values

The output also includes p-values.

- `bill_length_mm`: not statistically significant in this model
- `bill_depth_mm`: not statistically significant at the common 0.05 level
- `flipper_length_mm`: highly statistically significant

For beginners, a simple interpretation is:

- smaller p-values suggest a predictor is helping explain the outcome
- larger p-values suggest the predictor may not be adding much once the other predictors are already in the model

R-squared

The R-squared value is about `0.7639`, which means the model explains about **76.4% of the variation** in penguin body mass.

That is a strong result for a simple teaching example. It tells us that these three measurements, especially flipper length, do a good job predicting body mass.

Step 4: Build the logistic regression model

The script fits this model:

```
sex_model <- glm(
  sex ~ bill_length_mm + bill_depth_mm + flipper_length_mm,
  data = penguins_clean,
  family = binomial
)
```

What is logistic regression?

Logistic regression is used when the outcome variable is a **category**, especially a category with two choices.

Here, the outcome is:

- `sex` with two categories: `female` and `male`

This model does **not** predict body mass. Instead, it predicts the **probability** that a penguin belongs to one category.

Because we coded the factor levels as `female` then `male`, the model is predicting the probability that a penguin is **male**.

Why do we use `glm(..., family = binomial)`?

- `glm()` stands for generalized linear model
- `family = binomial` tells R we are fitting a logistic regression for a two-category outcome

Results of the logistic regression

When the script was run, the model produced these key coefficient estimates:

- Intercept: about `-61.69`
- `bill_length_mm`: about `0.098`
- `bill_depth_mm`: about `1.454`
- `flipper_length_mm`: about `0.162`

All three predictors had statistically significant p-values in this model.

How do we interpret logistic regression coefficients?

This is usually the hardest part for first-time learners.

In logistic regression, the coefficients do **not** directly tell us how much the predicted probability changes by 1 unit. Instead, they tell us how the predictor changes the **log-odds** of the outcome.

That sounds abstract, so here is the beginner-friendly version:

- positive coefficients increase the chance of being predicted as `male`
- negative coefficients decrease the chance of being predicted as `male`

Since all three coefficients are positive here, larger values of these body measurements are associated with a higher probability that the penguin is male.

Step 5: Create predictions

The script then creates a new dataset with model predictions:

```
penguin_predictions <- penguins_clean |>
  mutate(
    predicted_body_mass_g = predict(body_mass_model, newdata =
penguins_clean),
    probability_male = predict(sex_model, newdata = penguins_clean, type =
"response"),
    predicted_sex = if_else(probability_male >= 0.5, "male", "female")
  )
```

What is happening here?

`predicted_body_mass_g`

This uses the linear regression model to calculate a predicted body mass for each penguin.

`probability_male`

This uses the logistic regression model to calculate the probability that each penguin is male.

This value will be between 0 and 1.

Examples:

- `0.90` means the model thinks there is a 90% chance the penguin is male
- `0.25` means the model thinks there is a 25% chance the penguin is male

`predicted_sex`

This turns the predicted probability into a final category:

- if probability is at least `0.5`, label it `"male"`
- otherwise, label it `"female"`

This is called a **classification rule**.

Example of the output

The first few rows looked like this when the script was run:

```
# A tibble: 6 × 8
  body_mass_g bill_length_mm bill_depth_mm flipper_length_mm sex
        3750           39.1          18.7               181 male
        3800           39.5          17.4               186 female
        3250           40.3          18.0               195 female
        3450           36.7          19.3               193 female
        3650           39.3          20.6               190 male
        3625           38.9          17.8               181 female
```

The script also adds:

- `predicted_body_mass_g`
- `probability_male`
- `predicted_sex`

Those extra columns let students compare:

- the actual body mass versus predicted body mass
- the actual sex versus predicted sex

What should students learn from this example?

This example teaches several important ideas:

1. Choose the model based on the type of outcome

Use **linear regression** when the outcome is numeric, like `body_mass_g`.

Use **logistic regression** when the outcome is categorical with two groups, like `sex`.

2. Clean the data before modeling

Missing values and inconsistent category formatting can cause problems. The cleaning step is part of the modeling process, not an optional extra.

3. Regression is about relationships and prediction

These models help us understand how penguin body measurements relate to:

- body mass
- probability of being male or female

4. Coefficients should be interpreted carefully

In linear regression, coefficients are usually easier to explain because they relate directly to changes in the predicted numeric outcome.

In logistic regression, coefficients are harder to interpret directly, so beginners should focus first on the direction of the relationship and the predicted probabilities.

A beginner-friendly summary of the results

Here is the short version:

- The linear regression model did a good job predicting `body_mass_g`
- `flipper_length_mm` was the strongest predictor of body mass
- The logistic regression model used the same body measurements to estimate whether a penguin was male or female
- In the logistic model, larger measurement values were associated with a higher probability of being male

How to run the script

From R or RStudio, run:

```
source("Penguins/Regressions/penguin_regressions.R")
```

If students are working from the project root in the terminal, they can also run:

```
Rscript "Penguins/Regressions/penguin_regressions.R"
```

Final takeaway

This example is useful because it shows that the same dataset can support more than one kind of model:

- one model predicts a **number**
- another model predicts a **category**

That is one of the most important ideas in introductory predictive modeling. The type of question you ask determines the kind of model you should use.